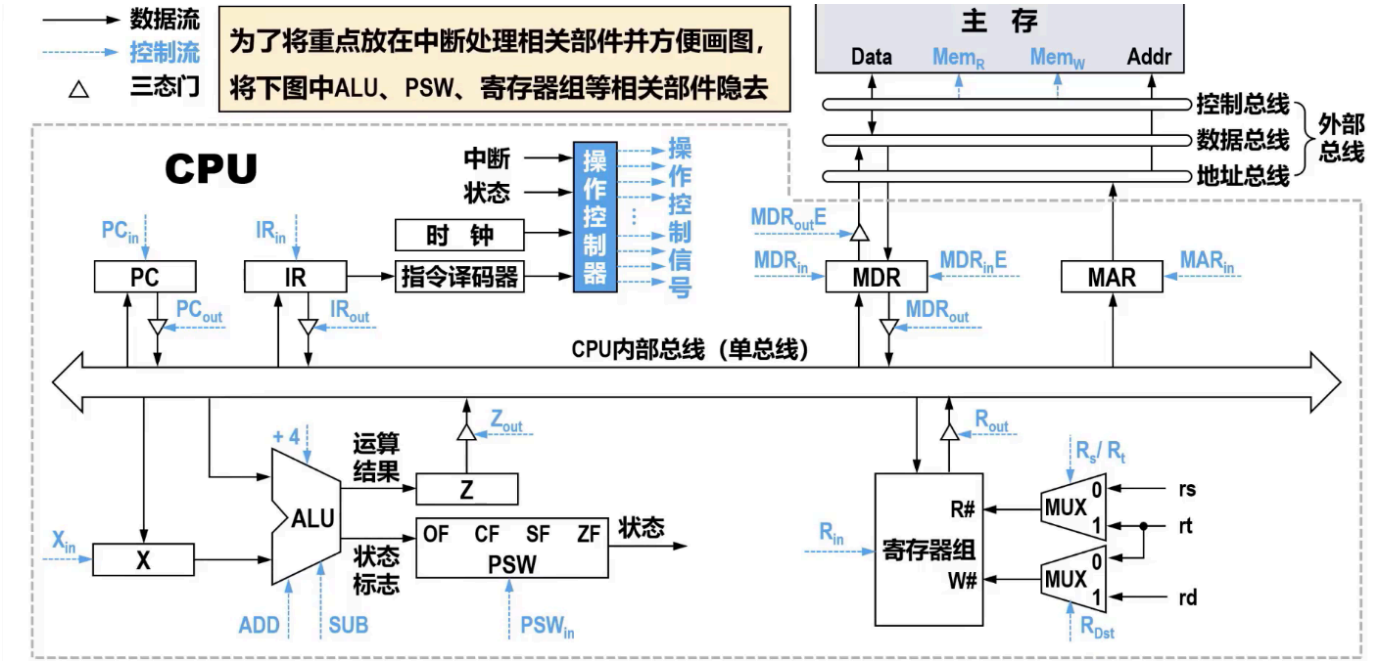
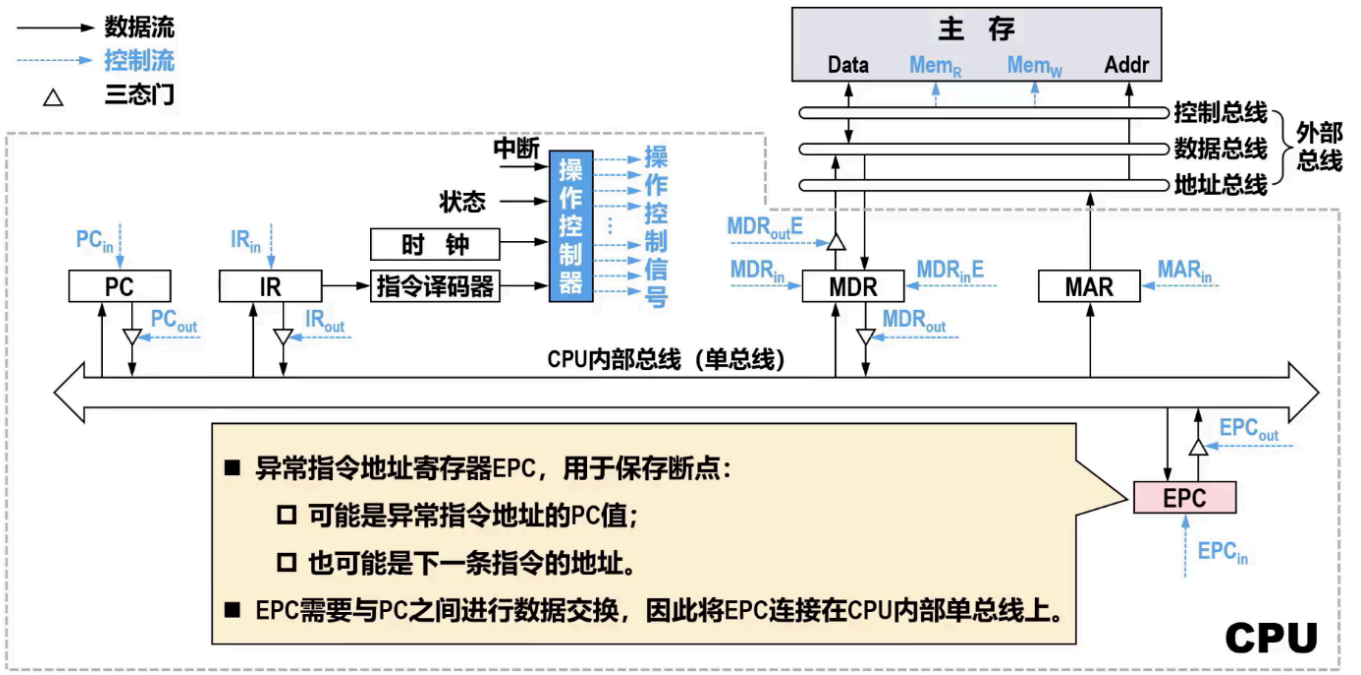
以单总线结构的数据通路为例，为方便绘图，省去了其中的一些部件，以下是原图：



一、EPC

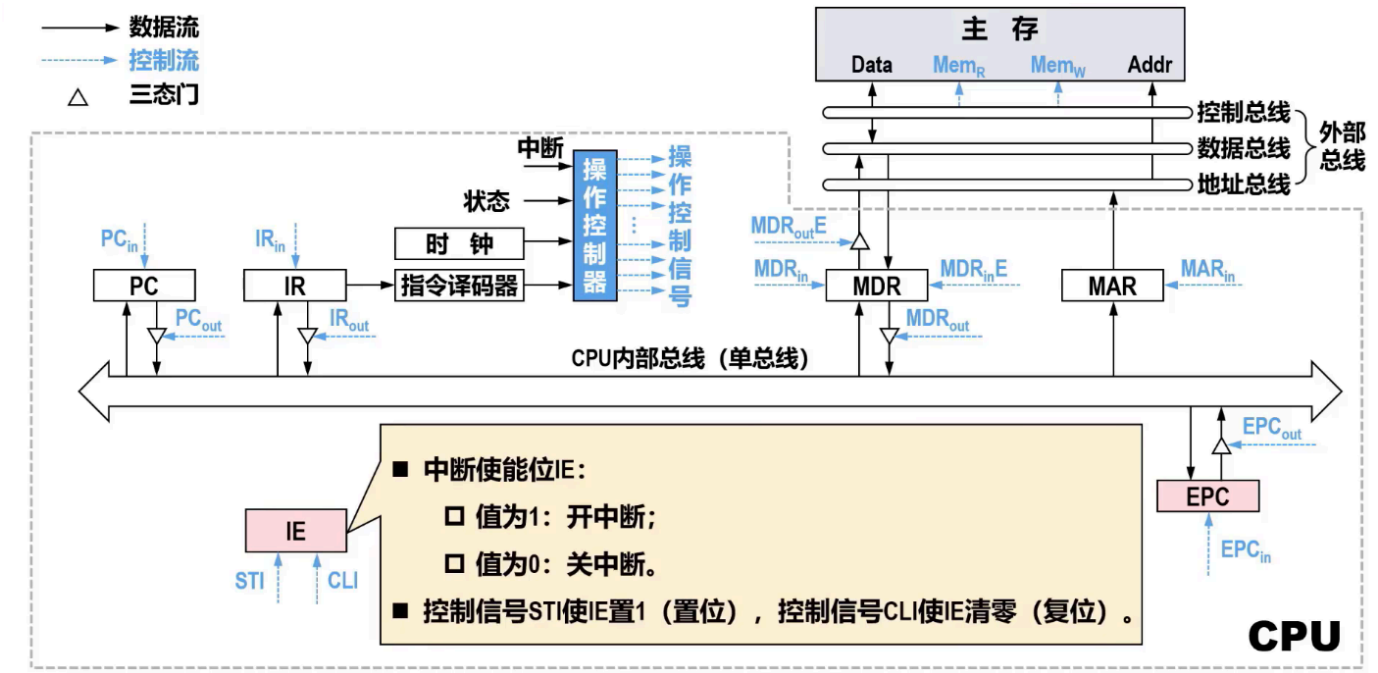


EPC：断点的专用保存处

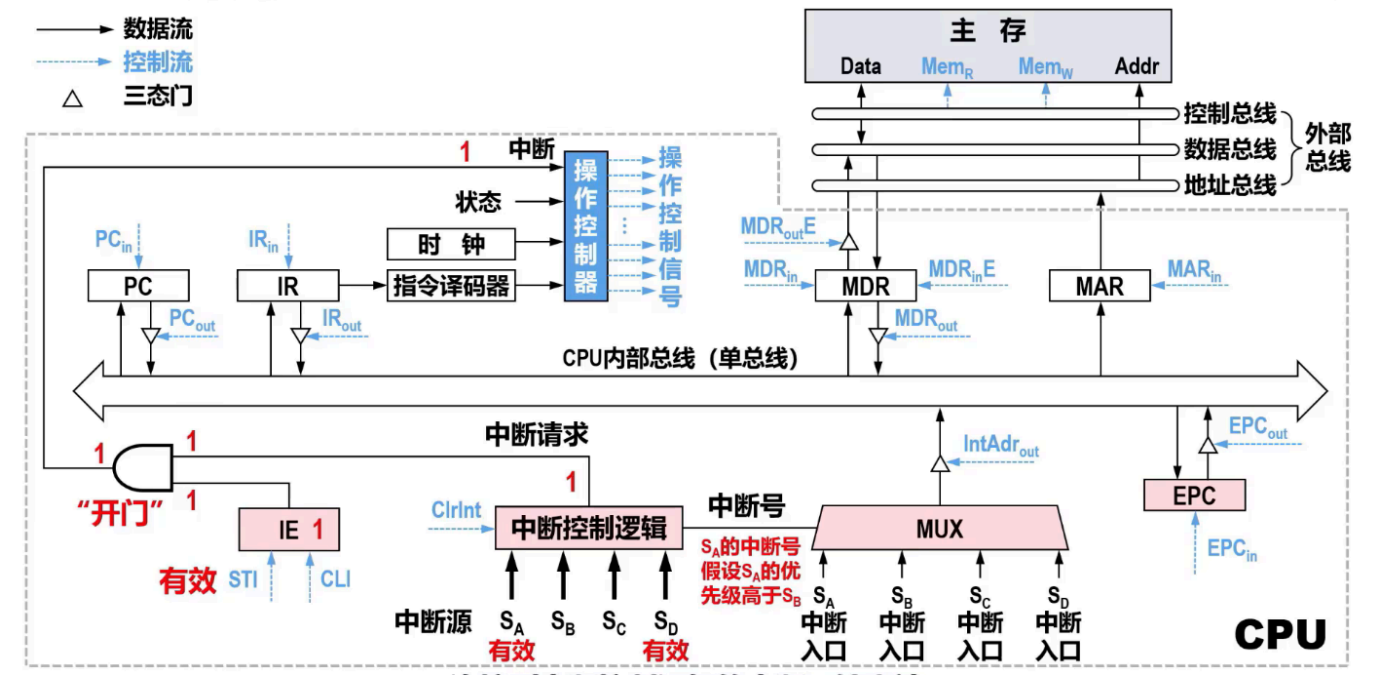
图示采用MIPS风格的断点保存方案——**硬件只把PC存入EPC寄存器**，不压栈。这与x86的"硬件自动压栈"形成对比：

- **EPC方案（MIPS风格）**：寄存器保存，速度快但容量有限，**嵌套中断时软件必须立即把EPC转存到内核栈**，否则新中断覆盖EPC导致原始断点丢失
- **堆栈方案（x86风格）**：硬件压栈，天然支持LIFO嵌套

二、IE



三、中断控制逻辑与MUX



中断控制逻辑 + MUX：向量中断的硬件实现

中断控制逻辑承担两个任务：**判优**（多个请求同时到达时按优先级选一个，图示假设SA优先级高于SB）和**编码**（把选中的中断源编码为中断号）

中断号送入MUX作为选择信号，MUX从多路常量输入（SA入口、SB入口、SC入口、SD入口）中选出对应的**中断服务程序入口地址**送入PC。这正是"向量中断"的硬件本质——**硬件直接给出入口地址，跳过软件查询**

与门"开门"：响应条件的硬件化

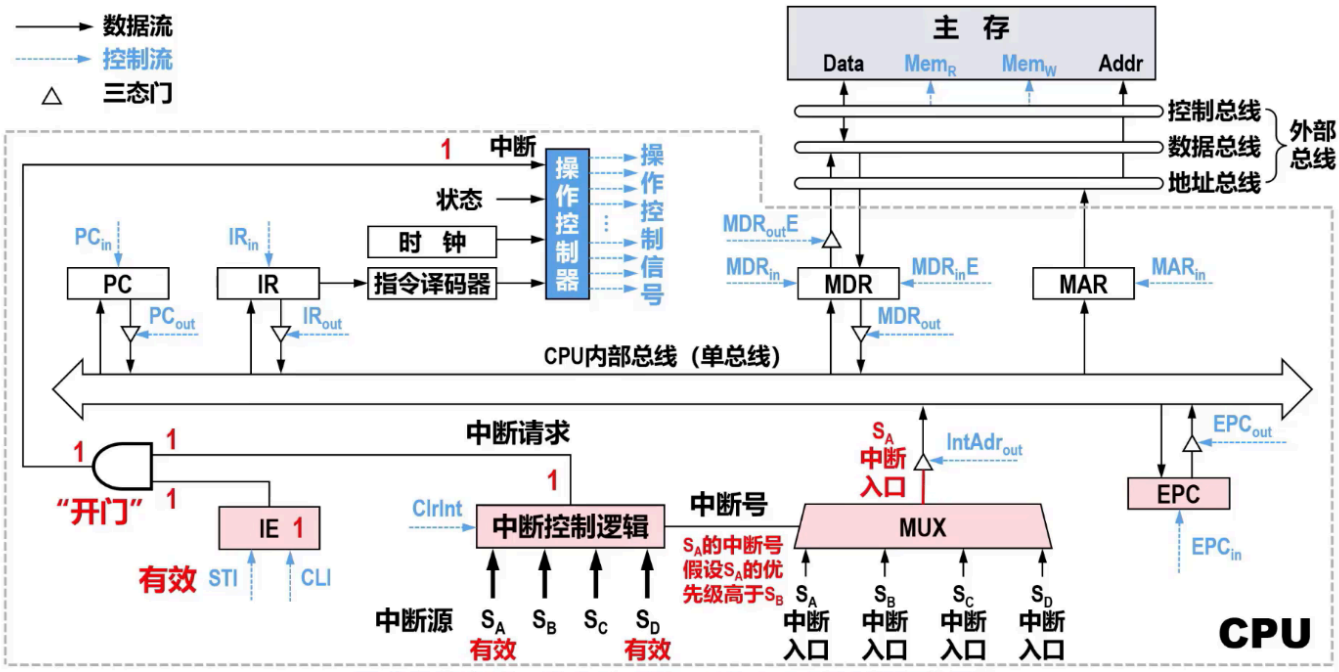
中断响应的三个必要条件中，有两条被这个与门电路直接固化到硬件里：

- **IE=1**：CPU处于开中断状态（异常和不可屏蔽中断不受此限制）

- **有未被屏蔽的有效中断请求**：图示中SA、SD有效，SB、SC无请求

与门输出为1时，操作控制器得知"下拍进入中断响应周期"，不再执行下一条指令的取指操作

五、完整面貌



六、一次完整的中断处理流程

阶段一 中断请求与判优（硬件自动）

外设（如SA）就绪后向中断控制逻辑发出请求信号，中断控制逻辑将其锁存为**中断请求触发器=1**；若同时有多个源请求，判优电路选出优先级最高者，编码为中断号送入MUX

阶段二 响应条件判断（硬件自动）

与门检查：IE=1 且 中断请求有效？若成立，输出1给操作控制器；CPU在**当前指令执行完毕的指令边界**进入中断响应周期

阶段三 中断隐指令（硬件一个机器周期内完成）

操作控制器在中断响应周期内自动发出控制信号，依次完成三件事，**全部硬件实现、不可被打断**：

```
① 关中断        : CLI使IE←0（防止后续步骤被打断）
② 保存断点      : EPC_in有效，PC → EPC（保存到专用寄存器）
③ 中断识别      : MUX根据中断号选SA入口 → PC_in有效，入口地址装入PC
   清除请求      : ClcInt有效，清除SA的中断请求触发器（避免重复响应）
```

中断响应周期结束时，**PC中已是SA的ISR第一条指令地址，EPC中保存着被打断程序的断点**

阶段四 中断服务程序执行（软件完成）

CPU从PC指向的ISR入口开始取指执行：

```
④ 保护现场      : 通用寄存器内容压栈（通过MDR_outE/MDR_inE访问内存）
⑤ 软件转存EPC  : EPC → 内存内核栈（MIPS风格必须做，防嵌套覆盖）
⑥ 开中断        : STI使IE←1（若支持嵌套，放行更高优先级）
⑦ 中断处理      : 执行具体服务逻辑（如读键盘、搬运数据）
⑧ 关中断        : CLI使IE←0
⑨ 恢复现场      : 从栈中弹回通用寄存器内容
⑩ 软件恢复EPC  : 从内核栈弹回原断点 → EPC
⑪ 中断返回      : ERET指令使EPC_out有效 → PC_in，断点回到PC
```

其中步骤④-⑩通过MDR的外部总线通路（图示MDRoutE、MDRinE、MARin）与主存交互，是软件可控的访存过程

阶段五 返回断点

中断返回指令（MIPS为eret）触发EPC_out控制信号，EPC中保存的断点经CPU内部总线送入PC，CPU回到被打断程序继续执行，一次中断处理完整结束